Les classes en Python

Introduction

```
>>> class Orange:
        pass

>>> Orange
<class '__main__.Orange'>
>>> type(Orange)
<class 'type'>
>>> orange1 = Orange()
>>> orange1
<__main__.Orange object at 0x00000205ABCF3F70>
>>> isinstance(orange1,Orange)
True
>>> dir(orange1)
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '
odule__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '_
>>> orange1.couleur = "orange"
>>> dir(orange1)
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '
odule__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '_
>>> orange1.couleur
'orange'
>>> orange1.__class__
<class '__main__.Orange'>
```

dir() : retourne les propriétés et méthodes d'un objet
__dict__ : les attributs et valeurs d'un objet créé dynamiquement
type(uneClasse) retourne 'type' parce qu'en Python, toutes les classes sont créées par la métaclasse type
type(objet) retourne la classe de l'objet: *type(orange1)* retourne *Orange*

... , 'couleur']

```
>>> orange1.__dict__   # attributs créés dynamiquement
{'couleur': 'orange'}
>>> orange1.gout = "délicieux"
>>> orange1.__dict__
{'couleur': 'orange', 'gout': 'délicieux'}
>>> orange2 = Orange()
>>> orange2.__dict__
{}
>>> del orange1.couleur
>>> orange1.__dict__
{'gout': 'délicieux'}
>>>
```

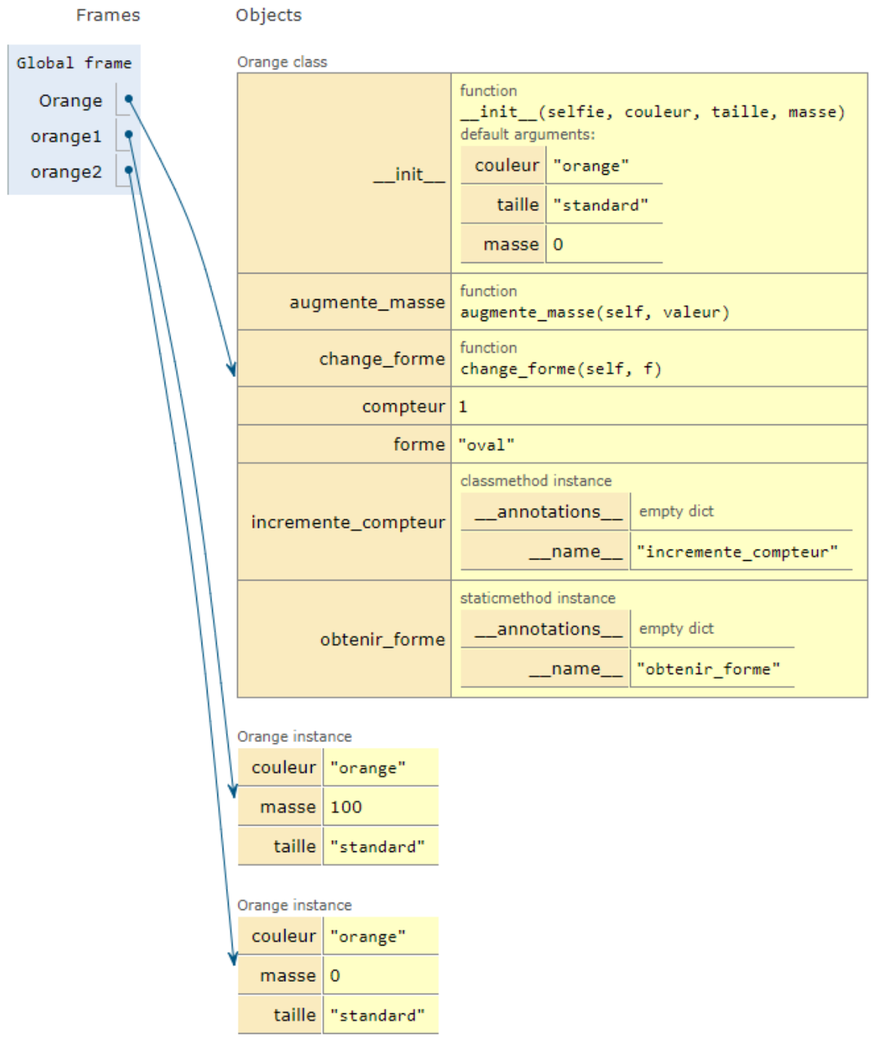
Classe dans un script

```python
# classes.py
class Orange:
    # attributs de classe, les mêmes pour tous
    forme = "rond"
    compteur = 0
    # méthodes d'instance, le premier argument est l'objet appelant
    def __init__(selfie, couleur = "orange", taille = "standard", masse = 0): # constructeur
        selfie.couleur = couleur # attribut d'instance
        selfie.taille = taille # attribut d'instance
        selfie.masse = masse # attribut d'instance
    def augmente_masse(self, valeur):
        self.masse += valeur # instance
    def change_forme(self, f):
        Orange.forme = f  # attribut de classe
    @classmethod # Utilisée pour des opérations qui affectent la classe dans son ensemble
    def incremente_compteur(cls): # cls représente la classe appelante
        cls.compteur += 1
    @staticmethod  # Utilisée pour des opérations qui sont logiquement liées à la classe
                   # mais qui ne nécessitent ni accès aux attributs d'instance ni accès
                   # aux attributs de classe.
    def obtenir_forme(): # Pas besoin de "self" ni "cls"
        return Orange.forme  # on peut quand même accéder à
                             # un attribut de classe de cette façon
if __name__ == "__main__":
    orange1 = Orange()
    orange2 = Orange()
    print("Attributs de classe :", orange1.forme, orange2.forme)  # rond rond
    print("Attributs d'instance :", orange1.masse, orange2.masse) # 0 0
    orange1.change_forme("oval")
    orange1.augmente_masse(100)
    print("Attributs de classe :", orange1.forme, orange2.forme)  # oval oval
    print("Attributs d'instance :", orange1.masse, orange2.masse) # 100 0

    Orange.incremente_compteur() # Méthode de classe
    print(Orange.compteur)  # Accès à la classe via "cls": 1

    print(Orange.obtenir_forme())  # Méthode statique: oval
```

Structure interne (Python Tutor)



Surcharge (d'opérateurs)

```python
# surcharge.py
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    # Surcharge de l'opérateur +
    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)
    # Surcharge de l'opérateur -
    def __sub__(self, other):
        return Point(self.x - other.x, self.y - other.y)
    # Surcharge de l'opérateur *
    def __mul__(self, scalar):
        return Point(self.x * scalar, self.y * scalar)
    # Surcharge de l'opérateur ==
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
    # Surcharge de l'opérateur >
    def __gt__(self, other):
        return (self.x**2 + self.y**2) > (other.x**2 + other.y**2)  # Comparaison par distance à l'origine
    # Représentation officielle de l'objet (utilisée par repr())
    def __str__(self): # un peu comme toString en Java, <__main__.Point object at 0x00000264649C7D30>
        return f"Point(x={self.x}, y={self.y})"
# Exemple d'utilisation
p1 = Point(2, 3)
p2 = Point(4, 1)
# Surcharge de +
print(p1 + p2)  # Affiche: Point(x=6, y=4)
# Surcharge de -
print(p1 - p2)  # Affiche: Point(x=-2, y=2)
# Surcharge de *
print(p1 * 3)   # Affiche: Point(x=6, y=9)
# Surcharge de ==
print(p1 == p2)  # Affiche: False
# Surcharge de >
print(p1 > p2)  # Affiche: True (car la distance de p1 à l'origine est plus grande)
```

Héritage

```python
# heritage.py
class Fruit:
    categorie = "alimentation"
    def __init__(self, saveur = None, forme = None):
        print("Je suis dans le constructeur de la classe Fruit")
        self.saveur = saveur
        self.forme = forme
        print("Je viens de créer self.saveur et self.forme pour un Fruit")
    def montre_conseil(self, type_fruit, conseil):
        print("Je suis dans la méthode .montre_conseil() de la "
              "classe Fruit\n")
        return (f"Instance {type_fruit}\n"
                f"saveur: {self.saveur}, forme: {self.forme}\n"
                f"conseil: {conseil}\n")
class Poire(Fruit):
    def __init__(self, saveur = None, forme = None):
        print("Je rentre dans le constructeur de Poire, et je vais appeler "
              "le constructeur de la classe mère Fruit !")
        Fruit.__init__(self, saveur, forme)
        print("Je termine le constructeur de Poire, les attributs sont :\n"
              f"self.saveur: {self.saveur}, self.forme: {self.forme}\n")
    def __str__(self): # On surcharge str qui est utilisé par print()
        print("Je rentre dans la méthode .__str__() de la classe Poire")
        print("Je vais lancer la méthode .montre_conseil() héritée "
              "de la classe Fruit")
        return self.montre_conseil("Poire", "Bon en croustade !")
class Kiwi(Fruit):
    def __init__(self, saveur = None, forme = None):
        self.saveur = saveur  # Je n'appel pas le constructeur parent
        self.forme = forme
    def __str__(self):
        return self.montre_conseil("Kiwi", "Bon en salade !")
if __name__ == "__main__":
    # On crée une poire et un kiwi
    poire = Poire(saveur = "juteuse", forme = "poire")
    print("Poire>>", poire)
    print("-------")
    kiwi = Kiwi(saveur = "douce", forme = "ballon")
    print("Kiwi>>", kiwi)
```

```
Je rentre dans le constructeur de Poire, et je vais appeler le constructeur de la classe mère Fruit !
Je suis dans le constructeur de la classe Fruit
Je viens de créer self.saveur et self.forme pour un Fruit
Je termine le constructeur de Poire, les attributs sont :
self.saveur: juteuse, self.forme: poire

Poire>> Je rentre dans la méthode .__str__() de la classe Poire
Je vais lancer la méthode .montre_conseil() héritée de la classe Fruit
Je suis dans la méthode .montre_conseil() de la classe Fruit

Instance Poire
saveur: juteuse, forme: poire
conseil: Bon en croustade !

-------
Kiwi>> Je suis dans la méthode .montre_conseil() de la classe Fruit

Instance Kiwi
saveur: douce, forme: ballon
conseil: Bon en salade !
```

Encapsulation

```python
# encapsulation.py
class Etudiant:
    def __init__(self, nom, matricule):
        # membre publique
        self.nom = nom
        # membre privé  __
        self.__matricule = matricule

    def affiche(self):
        print("Détails de l'étudant:", self.nom, self.__matricule)

    # getter
    def get_matricule(self):
        return self.__matricule

    # setter
    def set_matricule(self, matricule):
        self.__matricule = matricule

jessica = Etudiant('Jessica', 1234567)
jessica.affiche()  # Détails de l'étudant: Jessica 1234567
print(jessica.nom) # Jessica
#print(jessica.__matricule)  # introuvable parce que privé
print(jessica._Etudiant__matricule)  # name mangling: 1234567
print(jessica.get_matricule()) # 1234567
jessica.nom = "Jessa"
jessica.set_matricule(7654321)
jessica.__matricule = 9999999  # possible mais sans conséquence
jessica.affiche() # Détails de l'étudant: Jessa 1234567
```

```
Détails de l'étudant: Jessica 1234567
Jessica
1234567
1234567
Détails de l'étudant: Jessa 7654321
```